



POLITECHNIKA
OPOLSKA

WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI I INFORMATYKI

PRACA DYPLOMOWA INŻYNIERSKA

**Aplikacja do tworzenia harmonogramów pracy dla
małych przedsiębiorstw**

Dawid Tkacz

nr albumu: 102584

kierunek: informatyka

studia stacjonarne I stopnia

promotor:

dr Serhii Lupenko, prof. PO

Opole, 2024

1. Spis treści	
2. Cel i zakres pracy	3
3. Wprowadzenie	4
4. Projekt	7
4.1. Wymagania funkcjonalne i нефункционалне	7
4.2. Diagram przypadków użycia	8
4.3. Diagram klas	11
5. Implementacja	13
5.1. Środowisko i narzędzia programistyczne	13
5.2. Wzorce projektowe i wzorce kodu używane w projekcie	13
5.2.1. Singleton	13
5.2.2. Guard Clause	15
5.3. Dodatkowe biblioteki i frameworki użyte w projekcie	16
5.3.1. Tailwind	16
5.3.2. OfficeOpenXml	17
5.4. Problemy implementacyjne	17
6. Interfejs użytkownika	18
6.1. Menu wyboru	18
6.2. Sekcja informacji o firmie	18
6.3. Sekcja informacji o pracownikach	20
6.4. Sekcja dni wolnych	21
6.5. Sekcja generowania grafiku	21
6.6. Pobrany plik Excel	23
7. Algorytmy	24
7.1. Ogólne podejście do generowania harmonogramu	24
7.2. Podział algorytmu na etapy	24
7.3. Zastosowanie optymalizacji w przydzielaniu pracowników	25
7.4. Analiza wydajności algorytmu generującego harmonogram	25
8. Podsumowanie	27
9. Bibliografia	28

2. Cel i zakres pracy

Celem niniejszej pracy jest opracowanie aplikacji umożliwiającej tworzenie harmonogramów pracy dla małych przedsiębiorstw. Aplikacja została zaprojektowana z myślą o firmach potrzebujących elastycznego narzędzia do planowania godzin pracy, uwzględniając strukturę firmy oraz potrzeby pracowników.

W ramach pracy wykonano interfejs użytkownika, który pozwala na wprowadzenie informacji o strukturze organizacyjnej firmy, takich jak dni tygodnia w których firma działa, godziny pracy oraz zapotrzebowanie na pracowników na poszczególnych stanowiskach. Dodatkowo, umożliwiono dodawanie danych dotyczących pracowników, w tym informacje na temat rodzaju umowy (umowa o pracę lub umowa zlecenie), minimalną liczbę godzin, którą dany pracownik powinien otrzymać na dany miesiąc oraz stanowiska na których pracownik może zostać obsadzony. Zaimplementowano również funkcję wprowadzania dni wolnych, które pracownicy chcieliby uzyskać.

Na podstawie wprowadzonych danych, aplikacja generuje harmonogram pracy na wybrany miesiąc. Harmonogram jest w formie tabeli w która za pomocą symboli obrazuje grafik. Istnieje również możliwość dostosowania symboli. Dodatkowo, wygenerowany harmonogram może zostać pobrany w formie pliku Excel, który poza nim zawiera, również informacje o liczbie godzin przypisanych do każdego pracownika. Po zmodyfikowaniu komórek tabeli arkusz automatycznie aktualizuje informacje o godzinach, np. po dodaniu dnia pracy pracownikowi.

3. Wprowadzenie

Planowanie i zarządzanie różnego rodzaju zasobami stało się kluczowym zagadnieniem w kontekście współczesnych wyzwań gospodarczych. Od czasów wczesnego przemysłu po czasy współczesne, metody planowania ewoluowały, przyjmując różne formy, które były odpowiedzią na potrzeby i ograniczenia poszczególnych sektorów. Wskutek rewolucji przemysłowej znacząco wzrosło zapotrzebowanie na automatyzację planowania zasobami. Wraz z gwałtownym wzrostem rozwoju informatyki w XX wieku pojawiło się wiele nowych możliwości na osiągnięcie tego celu, pojawiły się bardziej złożone metody planowania, które łączą algorytmy matematyczne z narzędziami informatycznymi, co pozwala na jeszcze bardziej efektywne zarządzanie zasobami. Tematyka planowania zyskała na znaczeniu w kontekście różnorodnych zastosowań, od przemysłu wytwórczego po usługi czy transport. Obecnie dzięki coraz szerszemu dostępowi do danych i zaawansowanej analityce, możliwe jest optymalizowanie procesów planowania, co przynosi wymierne korzyści zarówno dla organizacji, jak i ich klientów.

Współczesne podejścia do planowania pracy łączą różnorodne metody i techniki, które pozwalają na optymalne wykorzystanie dostępnych zasobów. W literaturze można znaleźć wiele informacji na temat algorytmów usprawniających planowanie takich jak algorytmy zachłanne [1], metaheurystyczne [2] bądź programowanie liniowe [1]. Te innowacje nie tylko zwiększają efektywność, ale również pozwalają na lepsze zarządzanie czasem i kosztami, co jest szczególnie ważne w kontekście konkurencji rynkowej. Ponadto, dynamiczny rozwój technologii komputerowej oraz dostępności dużych zbiorów danych przyczyniają się do ciągłej ewolucji technik planowania, umożliwiając organizacjom lepsze prognozowanie oraz podejmowanie decyzji na podstawie rzetelnych analiz. W obliczu globalizacji i rosnącej konkurencji na rynku, efektywne planowanie staje się kluczowym elementem sukcesu wielu organizacji. Dzięki temu mogą dostosować się do zmieniających się warunków i potrzeb klientów.

Współczesna technologia umożliwia tworzenie zaawansowanych, wydajnych aplikacji internetowych, które znajdują zastosowanie między innymi w tworzeniu i zarządzaniu harmonogramami. Technologie takie jak Blazor WebAssembly, będącą częścią frameworku .NET umożliwia tworzenie aplikacji internetowych uruchamianych po stronie klienta bez konieczności przetwarzania dużych ilości kodu na serwerze. W Blazor WebAssembly kod C# jest kompilowany do WASM [3], co pozwala na uruchamianie go bezpośrednio w przeglądarce użytkownika bez użycia JavaScript. Technologia ta pozwala na wygodne przechowywanie danych tymczasowych w przeglądarce za pomocą LocalStorage [4] lub nawet klasy C# z użyciem wzorca projektowego Singleton [5]. Algorytmy napisane w tym języku takie jak na przykład algorytm do generowania harmonogramu może zostać odpalony po stronie klienta bez konieczności wysyłania zapytań do serwera, co pozwala na używanie aplikacji nawet w trybie offline. Blazor tak jak wiele innych frameworków front-endowych pozwala używać komponentów, które są podstawową jednostką budującą interfejs użytkownika. Każdy komponent składa się z kodu C# oraz kodu HTML i może być łatwo ponownie użyty w ponownie w różnych miejscach aplikacji. Takie podejście ułatwia tworzenie modularnych elementów, co sprzyja lepszej organizacji i utrzymaniu kodu.

Tworzenie harmonogramu pracy często może być skomplikowane z uwagi na ilość zmiennych które trzeba wziąć pod uwagę. Jednym z kluczowych problemów jest konieczność stworzenia grafiku spełniającego wszystkie wymogi podczas gdy dostępność pracowników zmienia się w zależności od przysługujących im dni wolnych, czy zmian zdrowotnych. Podczas układania harmonogramu trzeba sobie radzić z nieprzewidywanymi sytuacjami oraz dostosować grafik do sezonowych zmian w zapotrzebowaniu na pracę. Dodatkowym wyzwaniem są ograniczenia wynikające z różnorodności umów, które regulują maksymalne godziny pracy oraz konieczność wzięcia pod uwagę pozycji pracy na których pracownik może pracować. Każdy z tych punktów wpływa na finalną strukturę harmonogramu. Układanie grafiku bez używania dedykowanych do tego narzędzi wymaga stałego monitorowania zmian, by uniknąć konfliktów i zapewnić zgodność z przepisami prawa pracy. Dla wielu przedsiębiorstw szczególnie tych działających w branży o zmiennej liczbie zleceń, zarządzanie harmonogramem stanowi kluczowe wyzwanie, które wpływa na wydajność oraz poziom zadowolenia pracowników.

Algorytmy optymalizacyjne [6] odgrywają kluczową rolę w procesach zarządzania i planowania. Dzięki automatyzacji zadań algorytmy te przyczyniają się do zwiększenia wydajności oraz ograniczenia potencjalnych błędów ludzkich. W efekcie, organizacje mogą skupić się na bardziej strategicznych celach, usprawniając codzienne działania. Algorytmy potrafią analizować dane dotyczące dostępności pracowników, ich preferencji oraz wymogów godzinowych, umożliwiając stworzenie grafiku, który minimalizuje nadgodziny i optymalizuje przydział godzin pracy. W ten sposób, firma może lepiej kontrolować koszty pracy. Wprowadzenie automatyzacji poprzez algorytmy przyczynia się zatem do znacznego usprawnienia procesów operacyjnych.

Wraz z postępującym rozwojem technologii tworzenia harmonogramów, narzędzia te mogą w przyszłości znacznie zwiększyć swoje możliwości, oferując jeszcze bardziej zaawansowane funkcje. Możliwość integracji z systemami sztucznej inteligencji i uczenia maszynowego stwarza perspektywę na dynamiczne dostosowywanie harmonogramów, które uwzględnia nie tylko obecne wymagania pracowników i dostępność, ale także przewidywane wzorce zachowań i potrzeby firmy. W miarę jak algorytmy stają się bardziej zaawansowane, tworzenie harmonogramów może wykraczać poza prostą optymalizację godzin pracy w kierunku bardziej złożonych analiz.

Aplikacja która została wykonana oferuje szeroki zakres funkcji, które ułatwiają zarządzanie danymi firmy i tworzenie odpowiednich harmonogramów pracy. Jest ona podzielona na podstrony gdzie każda z nich odpowiada za coś innego. W sekcji dotyczącej ustawień firmy użytkownik może zdefiniować dni w których firma działa oraz godziny rozpoczęcia i zakończenia pracy dla wszystkich dni bądź jeśli jest potrzeba to również dla każdego dnia z osobna. Poza tym można podać stanowiska na jakich pracują pracownicy oraz przypisać je do odpowiednich dni, co pozwala na precyzyjne dostosowanie harmonogramu do specyficznych potrzeb organizacji. W sekcji drugiej znajduje się moduł zarządzania danymi związanymi z pracownikami. Użytkownik może dodawać informacje o pracownikach, w tym identyfikator pracownika, rodzaj umowy oraz minimalną liczbę godzin w miesiącu która jest jedną z kluczowych danych na podstawie których grafik jest generowany. Dodatkowo może określić, na jakich stanowiskach dany pracownik może być przydzielony do pracy, aby umożliwić dopasowanie grafiku od indywidualnych kompetencji pracowników. Aplikacja umożliwia również dodawanie dni wolnych

zgłaszanych przez pracowników za co odpowiada kolejna sekcja strony. Finalna zakładka odpowiada za dostosowanie oraz generowanie finalnego harmonogramu. Należy w niej podać miesiąc oraz rok dla którego zostanie stworzony grafik, co pozwoli na wygenerowanie go z odpowiednią ilością dni. Jest również możliwość dodania dni miesiąca w których firma nie pracuje, co może być przydatne w przypadku świąt lub wewnętrznych założeń firmy. Harmonogram jest prezentowany w formie przejrzystej tabeli w której za pomocą symboli można zobaczyć, czy dany pracownik ma dzień roboczy, wolne czy też przypada święto. Symbole te są rozmieszczone w odpowiednich komórkach tabeli, co umożliwia szybkie rozpoznanie statusu każdego dnia bez konieczności dodatkowych objaśnień. Aplikacja daje możliwość dostosowania tych symboli według preferencji użytkownika, co pozwala firmie na personalizację grafiku zgodnie z jej indywidualnymi potrzebami oraz ułatwia interpretację harmonogramu przez pracowników. Dzięki temu narzędziu tabela jest nie tylko funkcjonalna, ale również wizualnie czytelna i intuicyjna w obsłudze. Na koniec, aplikacja umożliwia eksportowanie wygenerowanego grafiku jako plik Excel. Plik ten zawiera nie tylko sam grafik, ale również szczegółowe informacje o liczbie godzin pracy przypisanych do każdego dnia oraz sumaryczne zestawienie godzin przepracowanych przez każdego pracownika w danym miesiącu. Co więcej, dzięki dynamicznym formułom, każda zmiana dokonana bezpośrednio w pliku na przykład dodanie dodatkowego dnia pracy dla danego pracownika powoduje automatyczne zaktualizowanie całkowitej liczby godzin, co wspomaga bieżące monitorowanie zgodności z wymogami czasu pracy.

4. Projekt

Projektowanie systemu obejmowało zdefiniowanie architektury, kluczowych modułów oraz zależności pomiędzy poszczególnymi komponentami. Na etapie projektowania szczególną uwagę poświęcono elastyczności i skalowalności aplikacji, aby umożliwić jej przyszłą rozbudowę i łatwe dostosowanie do zmieniających się potrzeb użytkowników.

4.1. Wymagania funkcjonalne i нефункционалне

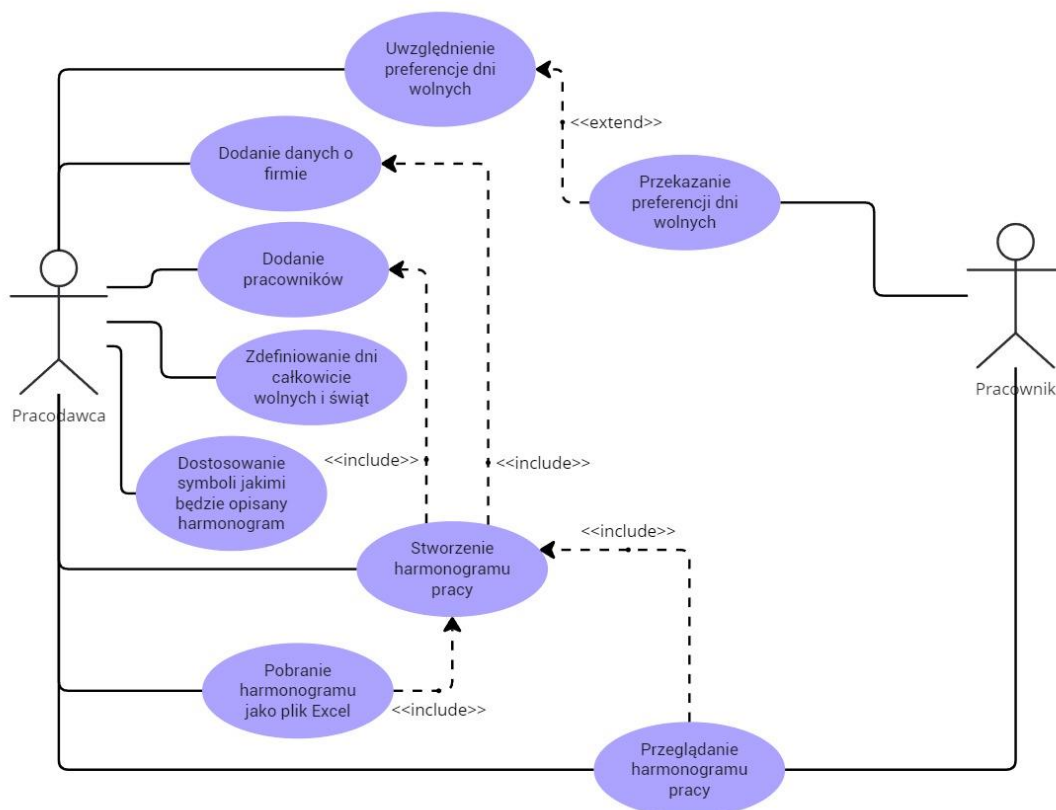
Wymagania funkcjonalne:

- Możliwość wprowadzania informacji o dniach i godzinach pracy firmy.
- Opcja dodawania stanowisk oraz przypisywania ich do odpowiednich dni.
- Dodawanie informacji o pracownikach, takich jak identyfikator, rodzaj umowy, minimalna liczba godzin w miesiącu oraz możliwe stanowiska.
- Funkcjonalność pozwalająca na rejestrację dni wolnych zgłaszanych przez pracowników.
- Tworzenie harmonogramu pracy na podstawie danych o firmie i pracownikach.
- Prezentacja grafiku w formie tabeli.
- Możliwość dostosowania symboli za pomocą których grafik jest przedstawiany.
- Opcja dodawania dni wolnych dla wszystkich pracowników.
- Eksport harmonogramu do pliku Excel, zawierającego szczegółowe informacje na temat godzin pracy.
- Automatyczna aktualizacja godzin pracy w pliku Excel po modyfikacji grafiku.

Wymagania нефункционалне:

- Intuicyjny interfejs użytkownika umożliwiający łatwe nawigowanie po aplikacji i szybki dostęp do kluczowych funkcji.
- Szybkie generowanie harmonogramów i płynne działanie aplikacji
- Możliwość obsługi większej liczby pracowników oraz zmiany godzin pracy i struktury organizacyjnej bez znaczącego spadku wydajności aplikacji.
- Możliwość korzystania z aplikacji na różnych urządzeniach i w różnych przeglądarkach.

4.2. Diagram przypadków użycia



Rys. 3.1. Diagram przypadków użycia

Przypadek użycia: Uwzględnienie preferencji dni wolnych

Aktor: Pracodawca

Opis: Dodanie pracownikom dni wolnych w zakładce „Employee preferences”.

Warunki wstępne: Minimum jeden pracownik przekazał swoje preferencje odnośnie dni wolnych.

Przebieg:

1. Pracodawca wybiera zakładkę „Employee preferences”.
2. W odpowiednim polu wpisuje numer dnia miesiąca.
 - a. Numer poprawny i dzień zostaje dodany.
 - b. Numer niepoprawny, program prosi o podanie poprawnego numeru.
3. Pracodawca chce dodać kolejny dzień.
 - a. Powrót do punktu 1.

Przypadek użycia: Dodanie danych o firmie

Aktor: Pracodawca

Opis: Wypełnienie formularza danymi firmy.

Warunki wstępne: Użytkownik wcisnął przycisk „Edit information” w zakładce „Company information”.

Przebieg:

1. Pracodawca podaje nazwę firmy.
2. Wybiera dni tygodnia w których firma pracuje.
3. Wpisuje nazwę pozycji w odpowiednim polu oraz klika przycisk „Add position”.
 - a. Przechodzi dalej.
 - b. Dodaje kolejną pozycję pracy.
4. Klika przycisk „Proceed”.
 - a. Niektóre dane nie spełniają wymagań.
 - i. Powrót do punktu 1.
 - b. Wszystkie dane spełniają wymagania, następuje przejście do następnej strony formularza.
5. Pracodawca podaje godziny w których firma pracuje dla każdego dnia.
6. Wybiera z listy rozwijalnej pozycję dla każdego dnia.
7. Klika przycisk „Save”.

Przypadek użycia: Dodanie pracownika

Aktor: Pracodawca

Opis: Dodanie informacji o konkretnym pracowniku.

Warunki wstępne: Użytkownik wcisnął przycisk „Add Employee” w zakładce „Add employee”.

Przebieg:

1. Pracodawca podaje identyfikator pracownika.
2. Wybiera rodzaj umowy.
3. Podaje minimalną liczbę godzin którą pracownik powinien uzyskać.
4. Wybiera pozycje na której pracownik może pracować i klika przycisk „Add position”.
 - a. Dodaje kolejną pozycję.
5. Wciska przycisk „Add employee”.
 - a. Wszystkie dane są poprawne i następuje zapisanie pracownika.
 - b. Niektóre dane nie spełniają wymagań i program prosi o ponowne uzupełnienie formularza.

Przypadek użycia: Zdefiniowanie dni całkowicie wolnych oraz świąt

Aktor: Pracodawca

Opis: Dodanie dni w które cała firma nie pracuje.

Warunki wstępne: Użytkownik jest w sekcji „Your schedule”.

Przebieg:

1. Pracodawca podaje numer dnia miesiąca w odpowiednim polu obok napisu „Holidays”.
2. Klika przycisk „Add”.
 - a. Numer spełnia kryteria i zostaje zapisany.
 - b. Numer nie spełnia wymagań i program prosi o ponowne podanie numeru.

Przypadek użycia: Dostosowanie symboli jakimi będzie opisany harmonogram

Aktor: Pracodawca

Opis: Dostosowanie symboli które będą tworzyć grafik.

Warunki wstępne: Użytkownik nacisnął przycisk „Adjust symbols” w sekcji „Your schedule”.

Przebieg:

1. Pracodawca wybiera symbol który na grafiku będzie oznaczać święto.
2. Wybiera symbol który na grafiku będzie oznaczać nieprzydzielony.
3. Wybiera symbol który na grafiku będzie oznaczać dzień wolny.
4. Klika przycisk „Save”
 - a. Wszystkie dane są poprawne i następuje zapisanie symboli.
 - b. Niektóre dane nie spełniają wymagań i program prosi o ponowne uzupełnienie formularza.

Przypadek użycia: Stworzenie harmonogramu pracy

Aktor: Pracodawca

Opis: Stworzenie finalnego harmonogramu na podstawie wcześniej podanych informacji.

Warunki wstępne: Dane o firmie oraz pracownikach zostały dodane, użytkownik jest w sekcji „Your schedule”.

Przebieg:

1. Pracodawca klika przycisk „Show schedule”.
 - a. Grafik się wygenerował.
 - b. Występuje błąd.

Przypadek użycia: Pobranie harmonogramu jako plik Excel

Aktor: Pracodawca

Opis: Pobranie pliku Excel zawierającego wygenerowany harmonogram oraz szczegółowe informacje na temat godzin pracowników.

Warunki wstępne: U jest w sekcji „Your schedule” i grafik jest już wygenerowany.

Przebieg:

1. Pracodawca klika przycisk „Download Excel file”.
 - a. Następuje pobieranie pliku.
 - b. Występuje błąd.

Przypadek użycia: Przeglądanie harmonogramu pracy

Aktor: Pracodawca, Pracownik

Opis: Użytkownik ma dostęp do wygenerowanego harmonogramu.

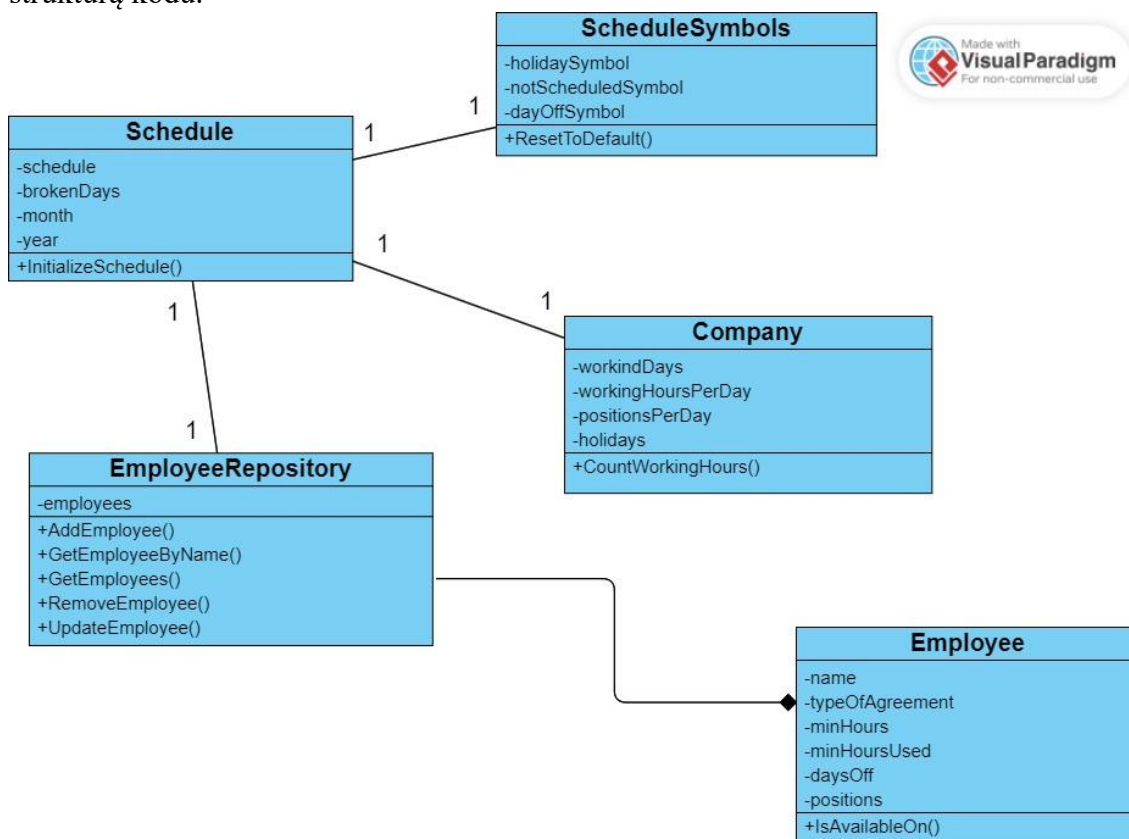
Warunki wstępne: Harmonogram został wygenerowany.

Przebieg:

1. Użytkownik przegląda harmonogram za pomocą aplikacji Excel.

4.3. Diagram klas

W celu zobrazowania struktury zależności pomiędzy elementami aplikacji stworzono diagram klas przedstawiający kluczowe klasy i powiązania między nimi. Diagram ten ukazuje, jak poszczególne moduły współpracują ze sobą aby osiągnąć założony efekt. Projekt został opracowany z myślą o podejściu obiektowym, co umożliwi lepsze zarządzanie strukturą kodu.



Powyższy diagram klas przedstawia logiczną strukturę aplikacji oraz relacje między jej kluczowymi komponentami. Aplikacja została zaprojektowana z użyciem podejścia obiektowego, co pozwala na modularność, łatwość rozbudowy oraz czytelność kodu.

W diagramie widoczna jest klasa `Schedule`, która pełni rolę centralnego komponentu odpowiedzialnego za generowanie harmonogramu pracy. Klasa ta korzysta z `EmployeeRepository`, które zarządza listą pracowników typu `Employee`. Relacja między tymi klasami została zrealizowana jako kompozycja, ponieważ po usunięciu `EmployeeRepository` tracimy wszystkie obiekty `Employee`. Dodatkowo `Schedule` jest powiązane z klasą `Company` oraz `ScheduleSymbols`. Obie te klasy zawierają dane, które klasa `Schedule` wykorzystuje do stworzenia grafiku.

5. Implementacja

W oparciu o wcześniej zdefiniowane wymagania funkcjonalne, przystąpiono do realizacji projektu. W trakcie implementacji wykorzystano nowoczesne technologie i narzędzia, które umożliwiły osiągnąć założone cele. W poniższych sekcjach opisano strukturę systemu, zastosowane frameworki oraz biblioteki, a także kluczowe elementy kodu źródłowego.

5.1. Środowisko i narzędzia programistyczne

Podstawowym środowiskiem pracy był Visual Studio Code, który jest popularnym edytorem kodu źródłowego, który charakteryzuje się dużą elastycznością i możliwością dostosowania do potrzeb programisty. Program wspiera wiele języków programowania, a dzięki licznym rozszerzeniom może on znacząco ułatwić pracę. Jego interfejs jest prosty i intuicyjny, co ułatwia pracę nad projektem i pozwala na szybką nawigację między plikami oraz różnymi funkcjami aplikacji.

Do zarządzania wersjami kodu oraz śledzenia historii zmian wykorzystano Git [7], czyli system kontroli wersji, który pozwala na śledzenie wszystkich modyfikacji w projekcie. Git umożliwia tworzenie gałęzi, co jest istotne podczas pracy nad nowymi funkcjonalnościami lub przy testowaniu różnych rozwiązań. Co istotne system umożliwia także łatwe cofanie zmian, co znacząco zwiększa bezpieczeństwo pracy i pozwala na szybkie odzyskanie poprzednich wersji kodu.

Dzięki zastosowaniu Visual Studio Code oraz Git, praca nad projektem była bardziej efektywna i zorganizowana. Możliwość synchronizacji zmian w kodzie oraz łatwego zarządzania wersjami projektu pozwoliła na bieżąco monitorować postępy.

5.2. Wzorce projektowe i wzorce kodu używane w projekcie

Wzorce projektowe oraz wzorce kodu zostały wykorzystane w celu poprawy struktury kodu, jego czytelności oraz ułatwienia przyszłej rozbudowy. Zastosowanie odpowiednich wzorców pozwala na tworzenie kodu bardziej modułowego i łatwiejszego w utrzymaniu. W niniejszym rozdziale opisano wzorce, które zostały użyte w aplikacji, takie jak Singleton do zarządzania stanem oraz Guard Clause do uproszczenia walidacji danych.

5.2.1. Singleton

Singleton to wzorzec projektowy, który gwarantuje, że dana klasa będzie miała tylko jedną instancję przez cały czas działania aplikacji oraz zapewnia globalny punkt dostępu do tej instancji. Powoduje to, że wartości atrybutów danej klasy, będą takie same niezależnie od tego w którym miejscu aplikacji wywołamy instancje klasy.

```

6 references
public class Schedule
{
    4 references
    private static Schedule? instance { get; set; }
    1 reference
    private static readonly object lockObject = new object();

    1 reference
    private Schedule()
    {
        schedule = new string[0, 0];
        brokenDays = new List<int>();
        brokenDaysPositions = new Dictionary<int, List<string>>();
        month = string.Empty;
        year = string.Empty;
    }

    1 reference
    public static Schedule Instance
    {
        get
        {
            if (instance == null)
            {
                lock (lockObject)
                {
                    if (instance == null)
                    {
                        instance = new Schedule();
                    }
                }
            }
            return instance;
        }
    }
}

```

Rys. 4.1. Przykład użycia wzorca Singleton

W powyższym kodzie widzimy przykład użycia wzorca Singleton w aplikacji. Na początku klasy tworzymy statyczne pole `instance`, którego typem jest klasa którą reprezentuje, przechowuje on referencje do jedynej instancji klasy. Wzorec ten pozwala na stworzenie prywatnego konstruktora co uniemożliwia stworzenie obiektu za pomocą klasycznego słowa kluczowego `new`. Dostęp do instancji zapewnia właściwość `Instance`, która stworzy instancje jeśli jeszcze takowa nie istnieje, lecz nie pozwoli na stworzenie kolejnej, a odwoła się do tej już istniejącej.

Klasyczne odwołanie się do instancji w kodzie powinno wyglądać w ten sposób:

```
Schedule schedule = Schedule.Instance;
```

Dodatkowo Blazor oferuje wbudowane wsparcie dla tego wzorca, co ułatwia zarządzanie stanem aplikacji. Można to osiągnąć poprzez zarejestrowanie klasy jako Singleton w pliku `Program.cs` za pomocą metody:

```
builder.Services.AddSingleton<AppState>();
```

Dzięki temu Blazor tworzy tylko jedną instancję klasy AppState, która jest dostępna globalnie w całej aplikacji.

```
19 references
public class AppState
{
    26 references
    public Company CompanyInstance { get; set; }
    3 references
    public Company CompanySecondShift { get; set; }
    15 references
    public EmployeesRepository EmployeesRepository { get; set; }
    10 references
    public Schedule schedule { get; set; }
    12 references
    public ScheduleSymbols scheduleSymbols { get; set; }

    0 references
    public AppState()
    {
        CompanyInstance = new Company();
        EmployeesRepository = new EmployeesRepository();
        schedule = Schedule.Instance;
        CompanySecondShift = new Company();
        scheduleSymbols = new ScheduleSymbols();
    }
}
```

Rys. 4.2. Klasa AppState w omawianym projekcie

Każdy atrybut tej klasy to inna klasa zarejestrowana jako Singleton. Dzięki tej funkcji jesteśmy w stanie bardzo łatwo zamienić zwykłą klasę na klasę, która pozwala na współdzielenie danych w różnych częściach aplikacji.

5.2.2. Guard Clause

Guard Clause [8] jest to wzorzec kodu używany w projekcie, który służy do walidacji warunków w metodach, aby szybko zakończyć ich działanie, gdy pewne warunki nie są spełnione. Dzięki niemu kod staje się bardziej czytelny i zwięzły. Pomaga on unikać niepotrzebnych zagnieżdżeń kodu.

```
2 references
private void AddDate(Employee employee)
{
    if (employee.SelectedDayOff ≤ 0 || employee.SelectedDayOff ≥ 32)
    {
        errorMessage = "Invalid date format";
        return;
    }

    errorMessage = null;
    employee.daysOff.Add(employee.SelectedDayOff);
}
```

Rys. 4.3. Przykład użycia Guard Clause

W powyższym przykładzie widać jak działa ten wzorzec. W przypadku nie spełnienia warunku z samego początku metody program przerwie jej działanie i wyświetli stosowny komunikat, a reszta kodu nie będzie niepotrzebnie przetwarzana.

```
2 references
private void AddDate(Employee employee)
{
    if (employee.SelectedDayOff > 0 && employee.SelectedDayOff < 32)
    {
        errorMessage = null;
        employee.daysOff.Add(employee.SelectedDayOff);
    }
    else
    {
        errorMessage = "Invalid date format";
    }
}
```

Rys. 4.4. Przykład jak mogłaby wyglądać metoda bez użycia Guard Clause

W wersji kodu z listingu 4.4 widzimy przykład metody która robi dokładnie to samo, ale jest napisana bez użycia wzorca kodu. Taki sposób implementacji utrudnia zrozumienie kodu, ponieważ ciężiej jest zauważyć warunek zakończenia działania funkcji oraz powoduje niepotrzebne zagnieżdżenie kodu. W przypadku gdyby funkcja była bardziej rozbudowana każdy warunek zakończenia generowałby dodatkowe zagnieżdżenie.

5.3. Dodatkowe biblioteki i frameworki użyte w projekcie

W projekcie zastosowano różnorodne biblioteki oraz frameworki, które znacząco usprawiły proces tworzenia aplikacji, zwiększając jej funkcjonalność oraz optymalizując wydajność. Dzięki ich wykorzystaniu możliwe było szybkie i efektywne zrealizowanie kluczowych zadań, takich jak stylizacja interfejsu użytkownika oraz generowanie plików Excel. Wybór odpowiednich narzędzi pozwolił na stworzenie nowoczesnej, responsywnej aplikacji, która spełnia wymagania związane z dynamicznym zarządzaniem harmonogramami pracy.

5.3.1. Tailwind

Tailwind CSS to nowoczesny framework do tworzenia stylów CSS oparty na podejściu utility-first, co oznacza, że zamiast definiowania stylów w oddzielnych arkuszach, programista używa predefiniowanych klas bezpośrednio w kodzie HTML. Dzięki temu możliwe jest szybkie tworzenie responsywnych interfejsów użytkownika.

Przykład stylizacji elementu użyty w aplikacji:

```
class="bg-[#661D15] text-red-500 text-2xl w-[1000px] h-[100px]
flex items-center justify-center ml-8 mt-8"
```

Stylizacja elementów odbywa się przy użyciu gotowych klas narzędziowych. Powyższy przykład określa kolor tła, wielość i kolor tekstu, wielkość elementu oraz umiejscowienie elementów wewnątrz.

5.3.2. OfficeOpenXml

OfficeOpenXml to biblioteka .NET umożliwiająca programistom pracę z plikami Excel (.xlsx) bez potrzeby użycia Microsoft Excel. Pozwala na tworzenie, odczytywanie i edytowanie arkuszy kalkulacyjnych w formacie OpenXML. Biblioteka jest szczególnie użyteczna w projektach, gdzie istnieje potrzeba dynamicznego generowania dokumentów Excel. Umożliwia ona zaawansowaną manipulację komórkami, wierszami i kolumnami, w tym także zastosowanie formuł, stylizacji czy wstawianie wykresów.

W aplikacji stworzono moduł generujący plik Excel z harmonogramem pracy dla pracowników. Wykorzystano klasę ExcelPackage [9] do tworzenia nowego arkusza oraz dynamicznego wypełniania go danymi. Dodatkowo, dzięki formułom SUMIF [10] arkusz automatycznie przelicza godziny pracy po naniesieniu zmian w wygenerowanym pliku, co upraszcza zarządzanie grafikiem.

5.4. Problemy implementacyjne

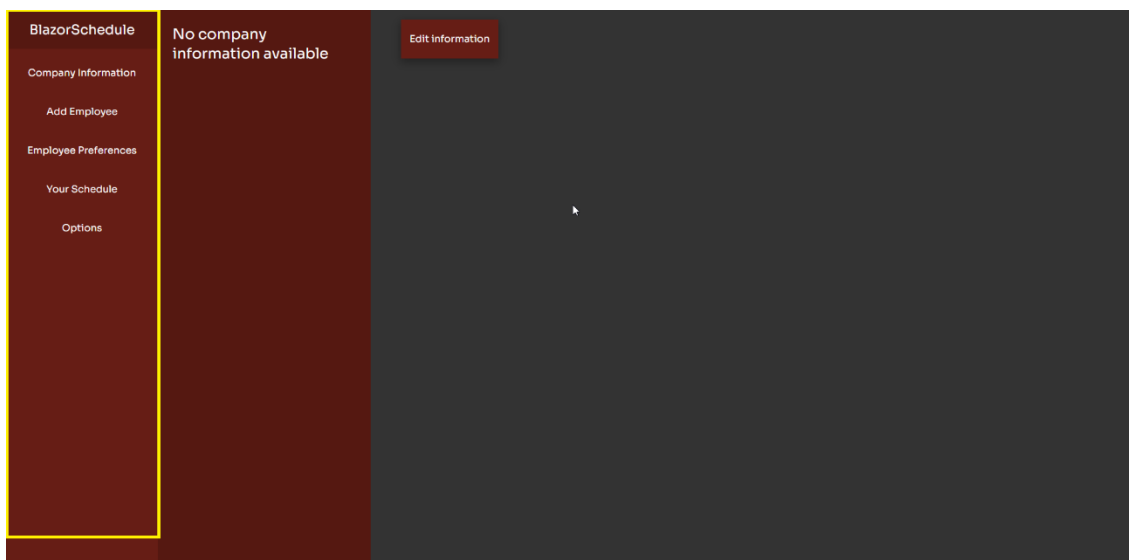
Jednym z kluczowych problemów napotkanych podczas implementacji algorytmu przydzielania pracowników do grafiku była początkowa koncepcja dotycząca kolejności wyboru pracowników, podczas sprawdzania czy spełniają wszystkie warunki by mogli zostać przypisani do danego dnia. Pierwotnie zakładano, że pracownicy będą brani pod uwagę w losowej kolejności, co miało na celu zapewnienie równomiernego rozłożenia obciążenia pracą. W praktyce jednak takie podejście sprawiało, że grafik był nieprzewidywalny oraz w niektórych przypadkach prowadziło to do nierównomiernego przydziału godzin lub niepełnego wykorzystania potencjału dostępnych pracowników. W wyniku analizy stwierdzono, że lepszym rozwiązaniem będzie przydzielanie pracowników w oparciu o liczbę godzin pozostałych do rozdzielenia. Algorytm został zmodyfikowany w taki sposób, aby priorytetowo przydzielać pracowników, którzy mają najwięcej niewykorzystanych godzin. Dzięki tej zmianie harmonogram stał się bardziej zbalansowany, a pracownicy otrzymali bardziej równomierne obciążenie pracą.

Podczas generowania grafiku, może się zdarzyć, że z podanych przez użytkownika danych nie ma możliwości stworzenia bezbłędnego grafiku. W takim przypadku grafik zostałby wyświetlony z błędem, na przykład któryś dzień mógłby nie mieć wystarczającej liczby przypisanych pracowników. Istnieje wtedy duże prawdopodobieństwo, że użytkownik przeoczy taki błąd i grafik nie zostanie poprawiony, aby tego uniknąć należało obsłużyć taki przypadek. Dodano do aplikacji funkcjonalność, która podczas generowania algorytmu analizuje czy każdy dzień ma odpowiednią liczbę przydzielonych pracowników, jeśli nie program zapisze informacje na temat błędu oraz podczas wyświetlania grafiku podświetli odpowiedni wiersz na czerwono i doda kolumnę, w której wyświetli brakujące stanowiska.

6. Interfejs użytkownika

W poniższym rozdziale przedstawiono szczegółowy opis poszczególnych widoków aplikacji, które umożliwiają użytkownikowi zarządzanie danymi firmy, pracownikami oraz harmonogramem pracy. Aplikacja została zaprojektowana w sposób intuicyjny, aby ułatwić proces planowania pracy. Składa się ona z kilku zakładek, które umożliwiają kompleksowe zarządzanie informacjami niezbędnymi do wygenerowania harmonogramu.

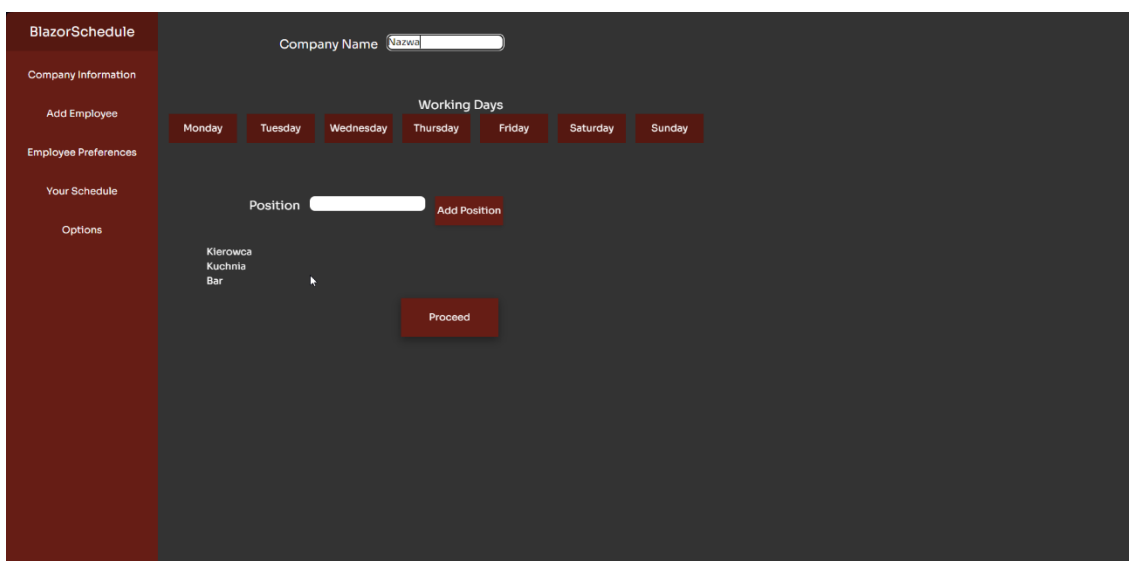
6.1. Menu wyboru



Rys. 5.1. Widok startowy

Na rysunku widoku startowego (Rys. 5.1) żółtym kolorem zaznaczono menu główne aplikacji z widocznymi przyciskami, które prowadzą do poszczególnych sekcji systemu. Panel ten znajduje się po lewej stronie ekranu niezmiennie przez cały czas używania aplikacji.

6.2. Sekcja informacji o firmie



Rys. 5.2. Widok dodawania informacji o firmie część 1

W widoku pierwszej części formularza służącego do zbierania informacji o firmie należy podać nazwę firmy, a następnie zaznaczyć kliknięciem dni w których firma działa. Po kliknięciu w poszczególny dzień kolor tła kafelka zmienia się na ciemniejszy na rysunku(Rys5.2) wszystkie dni są zaznaczone. Następnie należy dodać wszystkie pozycje na których będą pracować pracownicy.

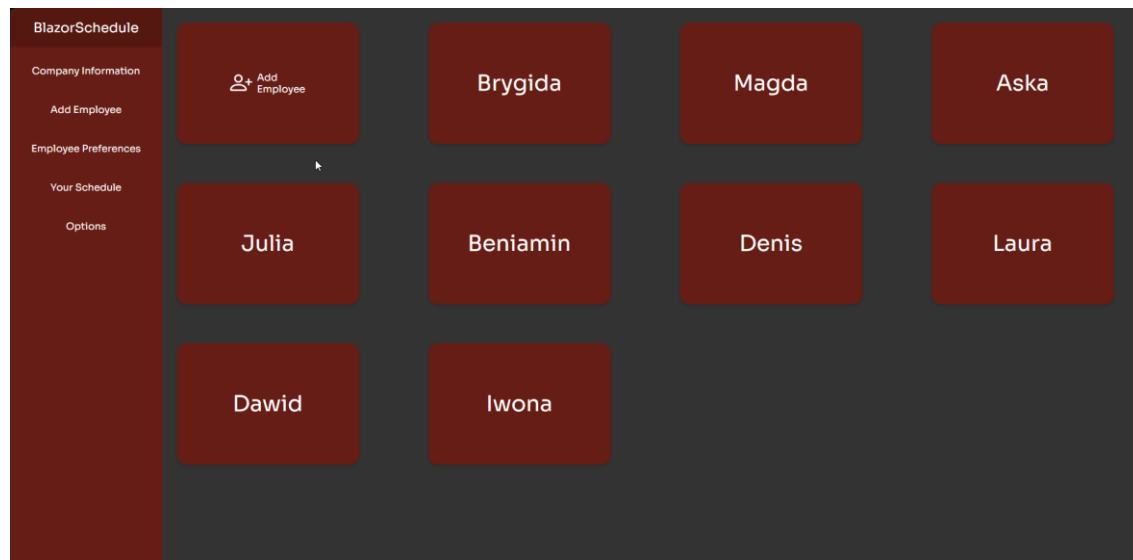
Rys. 5.3. Widok dodawania informacji o firmie część 2

Druga część formularza wykorzystuje informacje podane w części pierwszej, dzięki czemu widzimy panele z dniami tygodnia które zostały wybrane wcześniej oraz można wybierać pozycje spośród tych które użytkownik podał w części pierwszej. Widok służy do dostosowania godzin pracy oraz pozycji dla każdego dnia. Po zaznaczeniu pola wyboru „For all days” wystarczy podać jedną godzinę i zostanie ona automatycznie przypisana do każdego dnia.

Rys. 5.4. Widok z podsumowaniem podanych informacji o firmie

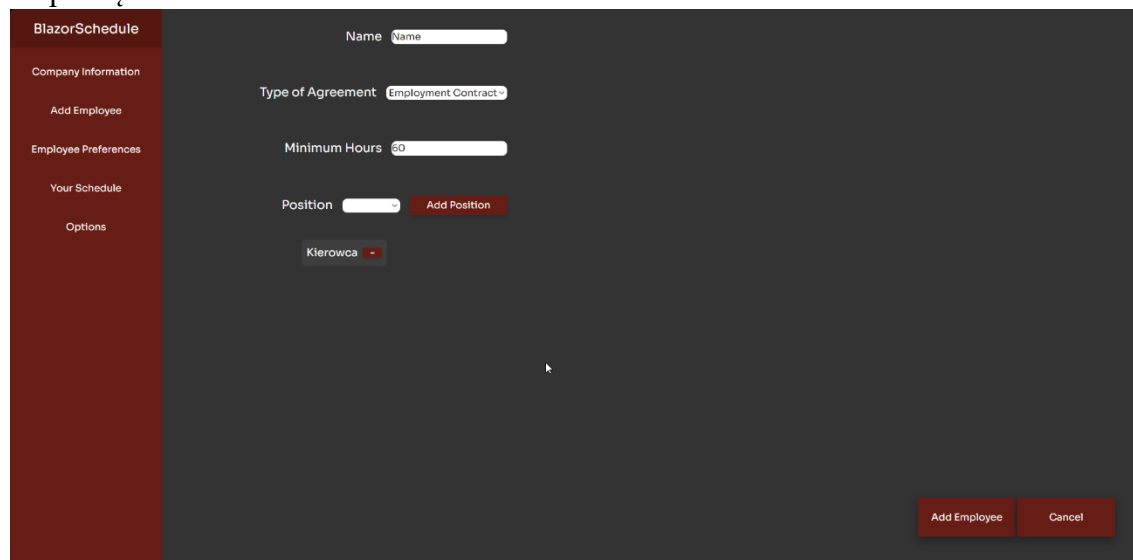
Po zapisaniu poprzedniego kroku aplikacja wraca do widoku pierwszego. Na prawo od głównego menu aplikacji znajduje się teraz podsumowanie z wypisanymi wszystkimi danymi podanymi w pierwszej oraz drugiej części formularza

6.3. Sekcja informacji o pracownikach



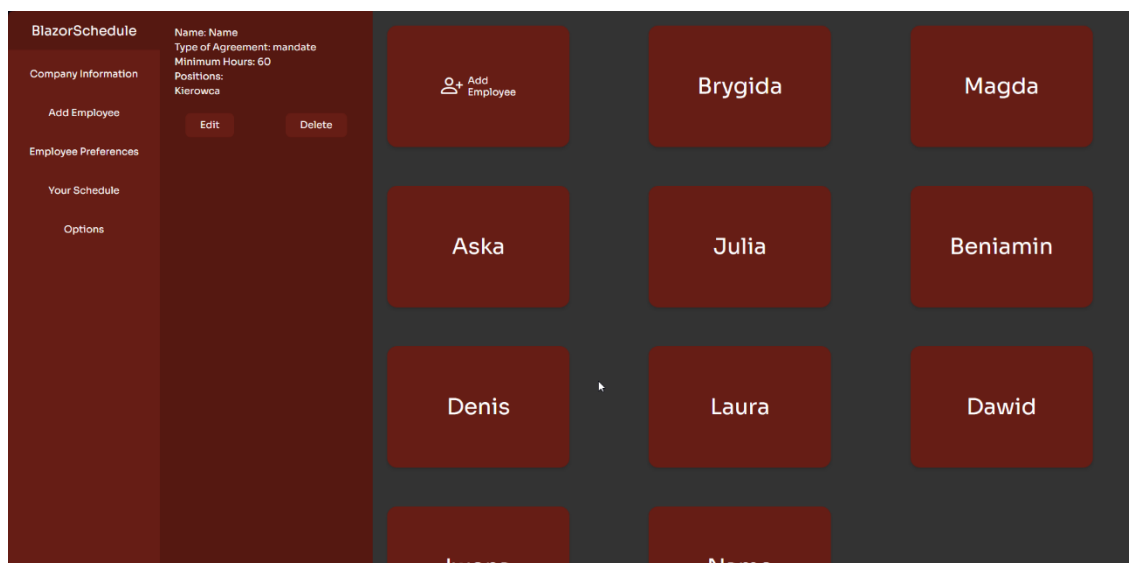
Rys. 5.5. Widok sekcji pracowników

Widok główny sekcji poświęconej pracownikom zawiera panele z nazwami pracowników oraz panel odpowiadający za dodanie nowego pracownika, który jest zawsze na początku.



Rys. 5.6. Widok dodawania pracownika

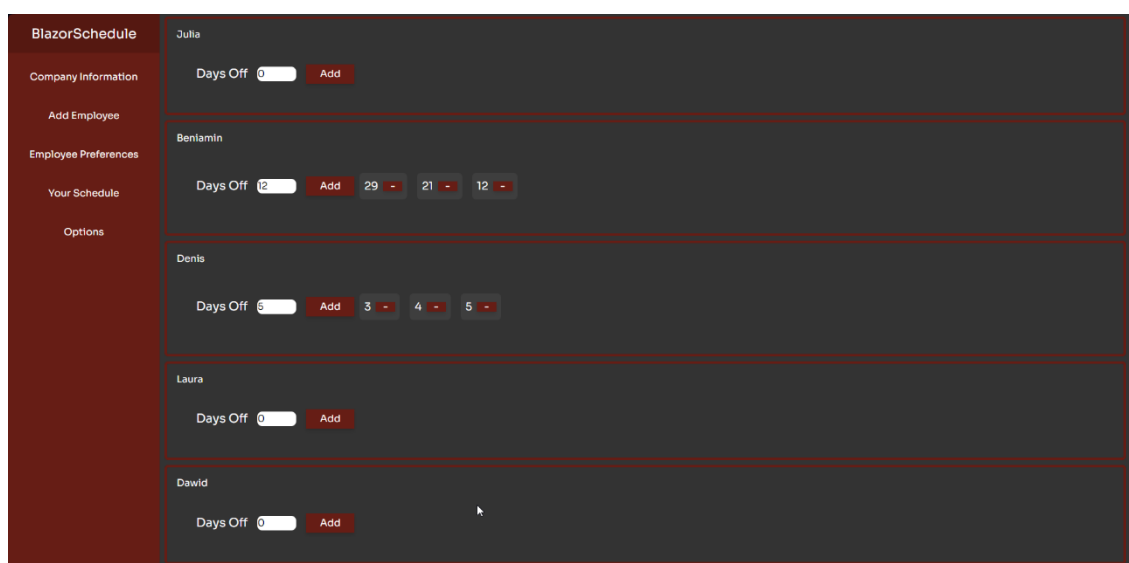
Widok dodawania pracowników pojawia się po kliknięciu panelu „Add employee” z poprzedniego widoku. Widzimy tutaj formularz, w którym należy podać identyfikator pracownika, wybrać rodzaj umowy oraz podać minimalną ilość godzin które algorytm powinien mu przydzielić na dany miesiąc. Poza tym należy wybrać pozycje z listy rozwijalnej. Istnieje możliwość dodania kilku pozycji jednemu pracownikowi.



Rys. 5.7. Widok szczegółów pracownika

Po naciśnięciu w panel z nazwą pracownika pojawia się po lewej stronie sekcja ze wszystkimi danymi jakie zostały podane dla wybranego pracownika. Istnieje również możliwość edycji oraz usunięcia.

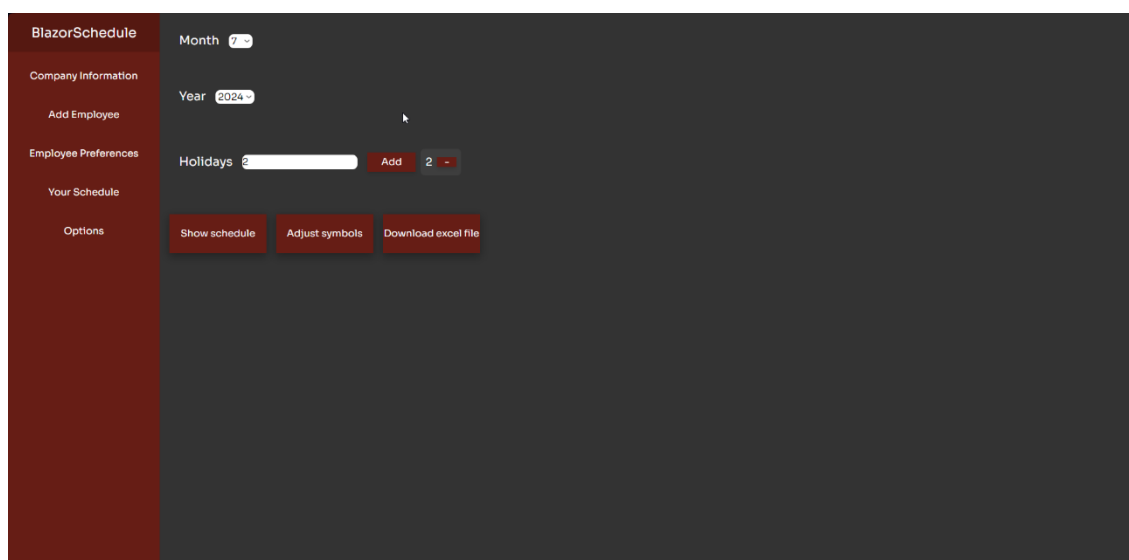
6.4. Sekcja dni wolnych



Rys. 5.8. Widok sekcji dni wolnych

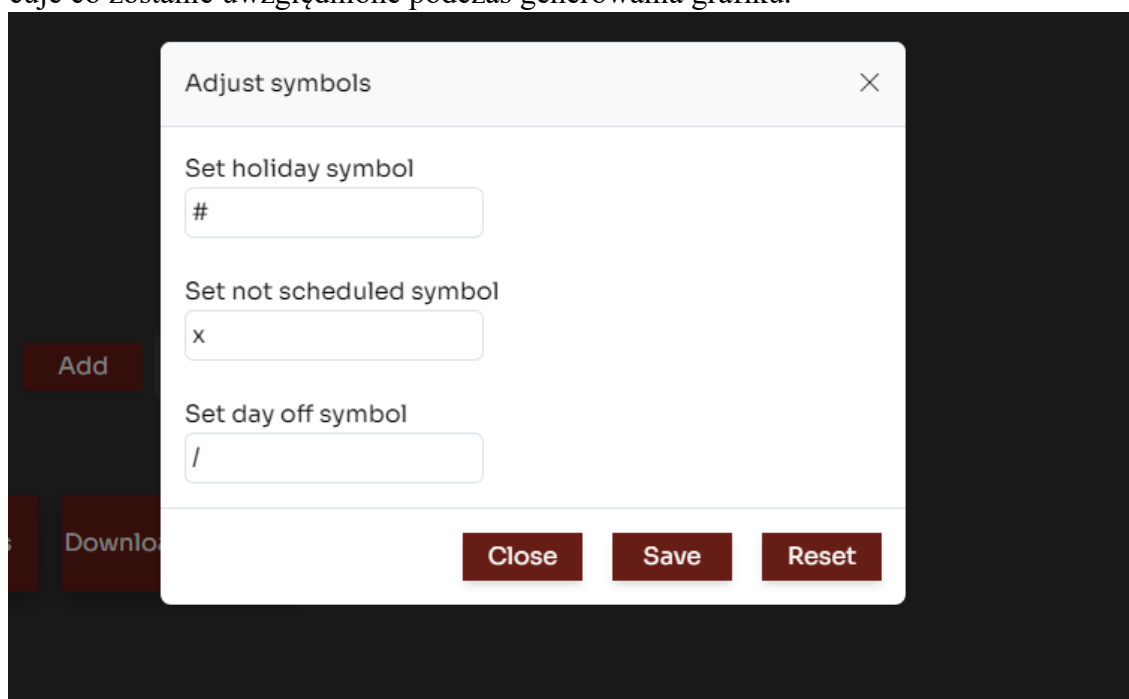
W tym widoku mamy możliwość dodania dni wolnych dla konkretnego pracownika, które algorytm uwzględni podczas tworzenia harmonogramu. Aby dodać dzień należy wpisać odpowiedni numer i pole tekstowe oraz wcisnąć przycisk „Add” lub Enter na klawiaturze.

6.5. Sekcja generowania grafiku



Rys. 5.9. Widok sekcji służącej do generowania grafiku

Widok sekcji służącej do generowania grafiku pozwala na podanie miesiąca oraz roku dla którego harmonogram zostanie wygenerowany oraz dni w które firma nie pracuje co zostanie uwzględnione podczas generowania grafiku.



Rys. 5.10. Pojawiające się okno służące do dostosowania symboli

Po wciśnięciu przycisku „Adjust symbols” pojawia się okno, którym można zmienić symbole którymi opisany będzie harmonogram. Dzięki przyciskowi „Reset” w każdym momencie można wrócić do podstawowych symboli.

7. Algorytmy

Wdrożone algorytmy odgrywają kluczową rolę w automatyzacji procesu przydzielania pracowników do zadań, uwzględniając zarówno dostępność pracowników jak i wymagania firmy. Celem tych rozwiązań było zapewnienie optymalnego wykorzystania zasobów ludzkich oraz minimalizacja czasu poświęconego na planowanie.

7.1. Ogólne podejście do generowania harmonogramu

Generowanie harmonogramu pracy w aplikacji wymaga zastosowania odpowiednich algorytmów, które automatyzują proces planowania przydziału pracowników do wymagań firmy uwzględniając przy tym dni robocze, dni wolne, ilość pracowników oraz ich kompetencje oraz inne wymagania firmy dotyczące przydzielania konkretnych dni pracownikom.

Finalny grafik przechowywany jest w dwuwymiarowej tablicy. Algorytm jest podzielony na kilka pomniejszych etapów, podczas których tablica jest aktualizowana. Późniejsze etapy wykorzystują efekt wcześniejszych oraz modyfikują tablice uwzględniając kolejne dane.

Główne metody używane w generowaniu harmonogramu to:

- MarkWorkingDaysInSchedule – oznacza dni robocze w harmonogramie
- AssignEmployeeDaysOff – przypisuje dni wolne poszczególnym pracownikom oraz święta
- AllocateEmployeesToPosition – przydziela pracowników do stanowisk w zależności od ich kompetencji, dostępności, preferencji i wymagań firmy

Każda z tych metod realizuje określone zadanie. Wspólnie składają się na pełny proces tworzenia harmonogramu.

7.2. Podział algorytmu na etapy

Algorytm generowania harmonogramu został podzielony na trzy główne etapy, z których każdy odpowiada za kluczowy element procesu planowania.

Pierwszy etap algorytmu polega na oznaczeniu dni roboczych w harmonogramie za co odpowiada metoda MarkWorkingDaysInSchedule. Metoda ta iteruje przez wszystkie dni miesiąca i sprawdza, które z nich są uznawane za dni robocze na podstawie ustawień zdefiniowanych przez użytkownika. Na podstawie listy workingDaysInt, która zawiera numery dni tygodnia metoda oznacza w harmonogramie te dni dostępne do pracy. Każdy dzień jest przypisany do odpowiedniej komórki tabeli schedule, a dzięki mechanizmowi automatycznego przechodzenia między dniami tygodnia, możliwe jest zachowanie cykliczności tygodniowej w harmonogramie.

Drugi etap obejmuje przypisanie dni wolnych oraz świąt do harmonogramu, co realizowane jest przez metodę AssignEmployeeDaysOff. Metoda ta uwzględnia dni świąteczne zdefiniowane przez firmę, jak i dni wolne zgłoszone indywidualnie przez pracowników. Jeśli dany dzień przypada na święto lub znajduje się na liście dni wolnych danego pracownika, wtedy komórka zostaje oznaczona odpowiednim symbolem.

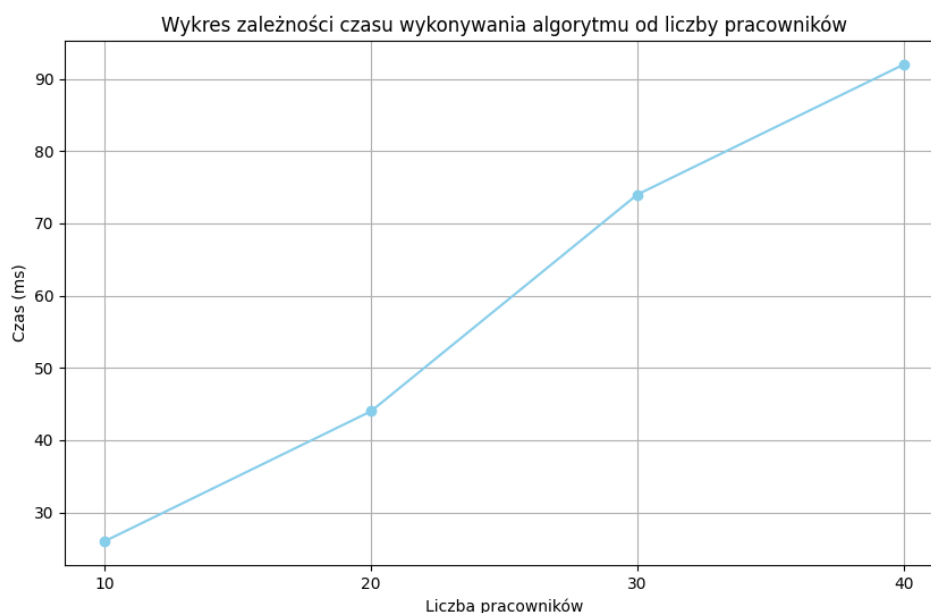
Ostatni etap algorytmu realizowany przez metodę `AllocateEmployeeToPositions`, zajmuje się przydzielaniem pracowników do stanowisk na podstawie ich dostępności, kwalifikacji oraz minimalnej liczby godzin, które muszą przepracować w miesiącu. Dla każdego dnia analizowane są wymagania firmy odnośnie obsady stanowisk, wykorzystując informacje zapisane w instancji `Company`. Pracownicy, którzy spełniają kryteria stanowiskowe, są dostępni w danym dniu i mają jeszcze godziny do przepracowania, są sortowani według liczby pozostałych godzin pracy w taki sposób, że priorytet mają ci, którzy mają najwięcej godzin do uzupełnienia. W ten sposób algorytm dąży do równomiernego rozłożenia dni pracy pomiędzy pracownikami. Jeśli nie uda się znaleźć pracownika spełniającego wymagania dla danego stanowiska, dzień zostaje oznaczony jako problematyczny, a nieobsadzone stanowiska są zapisywane do listy `brokenDaysPositions` w celu ułatwienia późniejszej ręcznej korekty.

7.3. Zastosowanie optymalizacji w przydzielaniu pracowników

W procesie generowania harmonogramu pracy zastosowano szereg technik optymalizacyjnych, aby zapewnić efektywne przydzielanie pracowników do stanowisk. Algorytm zawarty w metodzie `AllocateEmployeeToPositions` został zaprojektowany w taki sposób, aby wszyscy pracownicy zostali przydzieleni do pracy w sposób jak najbardziej optymalny. Optymalizacja polega na sortowaniu pracowników według liczby pozostałych godzin pracy przed rozpoczęciem kolejnych kroków algorytmu. Następnie sprawdzane jest czy pracownik spełnia wszystkie wymagania do przydzielenia w dany dzień, jeśli nie sprawdzany jest kolejny pracownik, aż do momentu kiedy warunki zostaną spełnione. Istotne jest, że kolejność sprawdzania pracowników jest taka jak kolejność posortowanej tablicy pracowników na podstawie liczby pozostałych godzin do rozdzielenia. Pozwala to równomiernie rozłożyć obciążenie pracą i minimalizować sytuacje, w których niektórzy pracownicy zostają przeciążeni, podczas gdy inni mają zbyt mało godzin.

7.4. Analiza wydajności algorytmu generującego harmonogram

W celu analizy wydajności algorytmu na podstawie zebranych danych stworzono wykres ilustrujący zależność czasu wykonywania algorytmu generującego grafik od liczby dodanych pracowników. Analiza ta pozwala ocenić wydajność algorytmu w zależności od wielkości zestawu danych, które program musi wziąć pod uwagę oraz zidentyfikować potencjalne problemy optymalizacyjne w przypadku wzrostu liczby pracowników.



Rys. 6.1. Wykres zależności czasu wykonywania algorytmu od liczby pracowników

Z powyższego wykresu można wywnioskować, że czas potrzebny do wygenerowania grafiku rośnie w sposób niemal liniowy wraz ze wzrostem liczby pracowników. Można z tego wywnioskować, że algorytm skalując się wraz z liczbą pracowników, nadal zachowuje przewidywalną wydajność. Każdorazowe dodanie nowych pracowników powoduje proporcjonalne zwiększenie czasu obliczeń. Jest to pozytywny wynik, ponieważ liniowa złożoność jest znacznie bardziej optymalna niż na przykład kwadratowa. Można przewidywać, że algorytm będzie sobie dobrze radził nawet przy generowaniu harmonogramu z większą liczbą pracowników bez drastycznego pogorszenia wydajności.

8. Podsumowanie

W ramach niniejszej pracy stworzono aplikację wspomagającą tworzenie harmonogramów pracy, skierowaną do firm poszukujących efektywnych narzędzi do zarządzania personelem. Aplikacja pozwala na zarządzanie danymi dotyczącymi firmy oraz pracowników. Na podstawie tych danych użytkownik może za pomocą jednego kliknięcia stworzyć i wyświetlić gotowy grafik dla danego miesiąca. Pozwala to znacząco zmniejszyć czas potrzebny na planowanie, a dzięki temu, że algorytm generuje harmonogram w sposób przewidywalny ryzyko błędu zostaje zminimalizowane. Istotną funkcjonalnością jest możliwość pobrania pliku Excel z wygenerowanym grafikiem. Plik jest skonfigurowany w taki sposób aby ułatwić ręczne zmiany w harmonogramie co może być niezbędne w niektórych przypadkach. Analiza wydajności algorytmu sugeruje, że aplikacja jest w stanie poradzić sobie z generowaniem grafiku w większych firmach posiadających dużą liczbę pracowników bez znaczącego spadku wydajności. Wykorzystanie technologii Blazor WebAssembly w połączeniu z frameworkiem Tailwind pozwoliło na stworzenie intuicyjnego oraz responsywnego menu aplikacji.

W przyszłości aplikacja może zostać wzbogacona o dodatkowe funkcje, które jeszcze bardziej usprawniają proces zarządzania harmonogramami. Możliwe jest dodanie do projektu systemów sztucznej inteligencji, które mogłyby być przydatne po wykonaniu algorytmu, przykładowo mogłyby w po napotkania konfliktów, które uniemożliwiają stworzenie grafiku zasugerować użytkownikowi zmiany w strukturze firmy lub odpowiednie dostosowanie pracowników w celu stworzenia harmonogramu niezawierającego błędów. Kolejną funkcjonalnością, która może zostać zaimplementowana jest baza danych przechowująca wcześniej wygenerowane harmonogramy oraz dodanie możliwości przesyłania preferencji odnośnie dni wolnych przez pracowników, co wymagałoby dodania systemu logowania oraz statusów użytkowników.

9. Bibliografia

- [1] Aditya Bhargava, *Algorytmy. Ilustrowany przewodnik*. Helion, 2017.
- [2] El-Ghazali Talbi, *Metaheuristics: From Design to Implementation*. Wiley, 2009.
- [3] Gerard Gallant, *WebAssembly in Action*. Manning, 2019.
- [4] Chris Sainty, *Blazor in Action*. Manning, 2022.
- [5] Gamma Erich, Helm Richard, Johnson Ralph, and Vlissides John, *Wzorce projektowe. Elementy oprogramowania obiektowego wielokrotnego użytku*. Helion, 2021.
- [6] Michael Mcmillan, *Data Structures and Algorithms Using C#*. Cambridge University Press, 2007.
- [7] Mariot Tsitoara, *Git i GitHub. Kontrola wersji, zarządzanie projektami i zasady pracy zespołowej*. Helion, 2022.
- [8] Refactoring.Guru, “Replace Nested Conditional with Guard Clauses.”, 2024
<https://refactoring.guru/pl/replace-nested-conditional-with-guard-clauses>
- [9] EPPlus Software, “ExcelPackage class documentation.”, 2024
<https://refactoring.guru/pl/replace-nested-conditional-with-guard-clauses>
- [10] Lambert Joan, *Excel 2021 i Microsoft 365. Krok po kroku*. APN Promise, 2022.